Industrial Internship Report on

Password Manager using Python

Prepared by

Parveena

Executive Summary

This report presents the work completed during the Python Development Internship organized by **upskill Campus** in collaboration with **UniConverge Technologies Pvt. Ltd. (UCT)**. The internship provided practical exposure to Python programming and software development through project-based learning.

The assigned project, **Password Manager using Python**, was developed to provide a secure and user-friendly solution for storing, managing, and retrieving login credentials. The application was built using **Python**, Tkinter for the graphical user interface, and SQLite as the database management system. Security was enhanced through SHA-256 hashing for master password authentication, while features such as password generation, search, update, delete, clipboard copy, and show/hide password functionality improved usability.

The project was developed using Python and built-in libraries. The application demonstrates CRUD (Create, Read, Update, Delete) operations, file handling, modular programming, and basic software testing.

The successful completion of this project strengthened my understanding of Python programming, GUI development, database management, software testing, debugging, and secure application development. It also enhanced my problem-solving ability, logical thinking, and practical software engineering skills. This internship served as a valuable learning experience by bridging the gap between academic knowledge and industry practices, preparing me for future software development projects and professional responsibilities.

Table of Contents

1 Preface

Industrial internships provide practical exposure to real-world software development and help students apply academic knowledge in professional environments. This internship, organized by **upskill Campus** and **The IoT Academy** in collaboration with **UniConverge Technologies Pvt. Ltd. (UCT)**, gave me the opportunity to develop a **Password Manager using Python**.

The project was developed using **Python,** Tkinter, and SQLite, with SHA-256 hashing for secure master password authentication. During the internship, I learned Python GUI development, database management, secure coding practices, software testing, debugging, and technical documentation.

I sincerely thank **UniConverge Technologies Pvt. Ltd. (UCT)**, **upskill Campus**, **The IoT Academy**, my mentors, and faculty members for their valuable guidance and continuous support throughout the internship. This experience has enhanced my technical skills and prepared me for future software development projects.

2. Introduction

Python is one of the most popular programming languages due to its simplicity, readability, and versatility. It is widely used in web development, automation, data analysis, artificial intelligence, cybersecurity, and desktop applications.

The purpose of this internship was to provide practical exposure to Python programming by developing a real-world project. The assigned project, **Password Manager**, demonstrates how Python can be used to build a secure application for managing user credentials. The application was built using **Python**, Tkinter, and SQLite to securely store and manage user credentials. It includes features such as master password authentication using SHA-256 hashing, password generation, search, update, delete, clipboard copy, and show/hide password functionality.

The project was completed by following the Software Development Life Cycle (SDLC), including requirement analysis, system design, implementation, testing, and documentation. This internship helped me strengthen my knowledge of Python programming, GUI development, database management, secure coding, and software testing while improving my problem-solving and software development skills.

2.1 About UniConverge Technologies Pvt. Ltd.

UniConverge Technologies Pvt. Ltd. (UCT) is a technology company specializing in Digital Transformation and industrial software solutions. The company works in areas such as Internet of Things (IoT), Cloud Computing, Artificial Intelligence, Machine Learning, Full Stack Development, Embedded Systems, and Industrial Automation. UCT develops scalable software solutions that help industries improve efficiency, automation, and productivity.

2.2 About upskill Campus

upskill Campus is a career development platform that provides students with internships, industrial projects, certification programs, and skill development opportunities. The platform focuses on bridging the gap between academic education and industry requirements by offering practical learning experiences through real-world projects.

2.3 About The IoT Academy

The IoT Academy is the EdTech division of UniConverge Technologies Pvt. Ltd. It provides professional training and certification programs in emerging technologies including Internet of Things (IoT), Full Stack Development, Artificial Intelligence, Machine Learning, Data Science, Cloud Computing, and Cyber Security. The academy aims to prepare students for industrial careers by combining theoretical concepts with practical implementation.

2.4 Objectives of this Internship Program

The main objectives of this internship were:

- To develop a secure Password Manager using Python.
- To gain practical experience in Python, Tkinter, and SQLite.
- To implement secure user authentication using SHA-256 hashing.
- To strengthen software development and database management skills.
- To understand the Software Development Life Cycle (SDLC).
- To improve problem-solving, debugging, and technical documentation skills.
- To use Git and GitHub for version control.

2.5 References

The following resources were used during the internship:

1. Python Official Documentation
2. Python Standard Library Documentation
3. Tkinter GUI Library
4. SQLite Database
5. SHA-256 Hashing Algorithm
6. GitHub Documentation
7. upskill Campus Learning Resources
8. UniConverge Technologies Training Materials
9. Online Python Tutorials

2.6 Glossary

Term	Meaning
Python	Programming language used to develop the application
SQLite	Lightweight database used to store password records
Tkinter	GUI library used to create the desktop interface
SHA-256	Cryptographic hashing algorithm used for master password authentication
Git	Version Control System
GitHub	Cloud-based code hosting platform
SLDC	Software Development Life Cycle
Repository	Collection of project files
README	Project documentation file

3. Problem Statement

3.1 Problem Statement

In today's digital era, users manage multiple online accounts for banking, social media, education, shopping, and other services. Remembering different strong passwords for each account is difficult, leading many users to reuse passwords or store them in insecure formats such as notebooks or plain text files. This increases the risk of unauthorized access and data breaches.

The objective of this project is to develop a **Password Manager** that provides a secure and efficient way to store, organize, and manage user credentials. The application authenticates users through a **master password** protected using SHA-256 hashing and stores password records in an SQLite database.

The system allows users to add, view, search, update, and delete password records. It also includes a strong password generator, clipboard copy feature, and password visibility control, making password management both secure and convenient.

The proposed solution aims to improve password security, simplify credential management, and provide a user-friendly desktop application developed using **Python**, Tkinter, and SQLite.

4. Existing and Proposed Solution

4.1 Existing Solution

Many people store passwords in notebooks, spreadsheets, or simple text files. Although these methods are easy to use, they have several disadvantages:

- Passwords can easily be lost or exposed.
- Manual searching takes more time.
- Updating or deleting records is inconvenient.
- No organized structure for managing multiple accounts.
- High possibility of entering duplicate or incorrect records.
- Difficult to maintain as the number of accounts increases.

Commercial password managers are available but often require paid subscriptions, internet access, or account registration, making them unsuitable for beginners learning Python programming.

4.2 Proposed Solution

The proposed solution is a **Password Manager Desktop Application** developed using **Python**, Tkinter, and SQLite. The application securely stores user credentials and protects access through SHA-256 hashed master password authentication.

- Key features of the proposed system include:
- Secure master password login
- Add, view, search, update, and delete password records
- Strong password generator
- Copy password to clipboard
- Show/Hide password functionality
- Automatic SQLite database creation
- Simple and user-friendly graphical interface

This solution provides a secure, lightweight, and offline password management system that improves both security and usability.

4.3 Features of the Proposed System

The Password Manager includes the following features:

- Secure master password authentication
- SHA-256 password hashing
- Automatic SQLite database creation
- Add password records
- View saved passwords
- Search password records
- Update existing passwords
- Delete password records
- Strong password generator
- Copy password to clipboard
- Show/Hide password option
- User-friendly Tkinter GUI
- Fast offline access
- Lightweight application
- Modular and scalable design

4.4 Advantages of the Proposed System

The developed application provides several advantages:

- Enhances password security
- Protects user credentials
- Easy to use
- Fast password retrieval
- Supports strong password creation
- Reduces password reuse
- Offline and privacy-focused
- Simple database management
- Saves time and effort
- Reliable data storage
- Easy maintenance
- Low system requirements
- Built with open-source technologies
- Suitable for personal use
- Easy to upgrade with new features

4.5 Technologies Used

Technology	Purpose
Python	Programming Language
Tkinter	Graphical User Interface (GUI)
SQLite	Database management
VS Code	Code development and debugging
Git & Github	Version Control and Code Repository

4.6 GitHub Repository

Repository Name

upskillCampus

GitHub Repository Link

upskillcampus/PasswordManager.py at main · parveena19/upskillcampus

---

4.7 Report Submission

Final Report PDF Link

upskillcampus/PasswordManager_Parveena_USC_UCT.pdf at main · parveena19/upskillcampus

---

4.8 Project Folder Structure

```
Password_Manager/
│
├── database/
│   └── password_manager.db
│
```

```
├── screenshots/
│
├── main.py
├── requirements.txt
├── README.md
├── .gitignore
```

4.9 Value Addition

1 This project not only fulfills the internship requirements but also provides practical experience in software development. The proposed Password Manager provides a secure, efficient, and user-friendly solution for managing digital credentials. Unlike traditional methods such as storing passwords in notebooks or plain text files, the application protects user access through SHA-256 hashed master password authentication and securely stores data in an SQLite database. It simplifies password management by offering features such as password generation, search, update, deletion, clipboard copy, and show/hide password functionality. The application operates completely offline, ensuring greater privacy and data security. Its modular design, lightweight architecture, and intuitive graphical interface make it reliable, easy to use, and suitable for future enhancements, thereby delivering a practical and secure password management solution.

.

5. Proposed Design / Model

5.1 System Overview

The proposed **Password Manager** is designed as a secure and user-friendly desktop application that enables users to store and manage their login credentials efficiently. The system follows a modular architecture, where each module performs a specific task to ensure better performance, maintainability, and security. The application is developed using **Python** for the backend logic, Tkinter for the graphical user interface, and SQLite for secure data storage.

The system consists of these major components:

- **Master Authentication Module:** Verifies the user's master password using SHA-256 hashing before granting access.
- **Password Management Module:** Allows users to add, view, search, update, and delete password records.
- **Password Generator Module:** Generates strong and random passwords for enhanced security.
- **Database Module:** Stores and retrieves password records securely using SQLite.
- **Clipboard Module:** Copies selected passwords to the clipboard for quick use.
- **User Interface Module:** Provides a simple and interactive graphical interface for easy navigation.

Working of the Proposed System

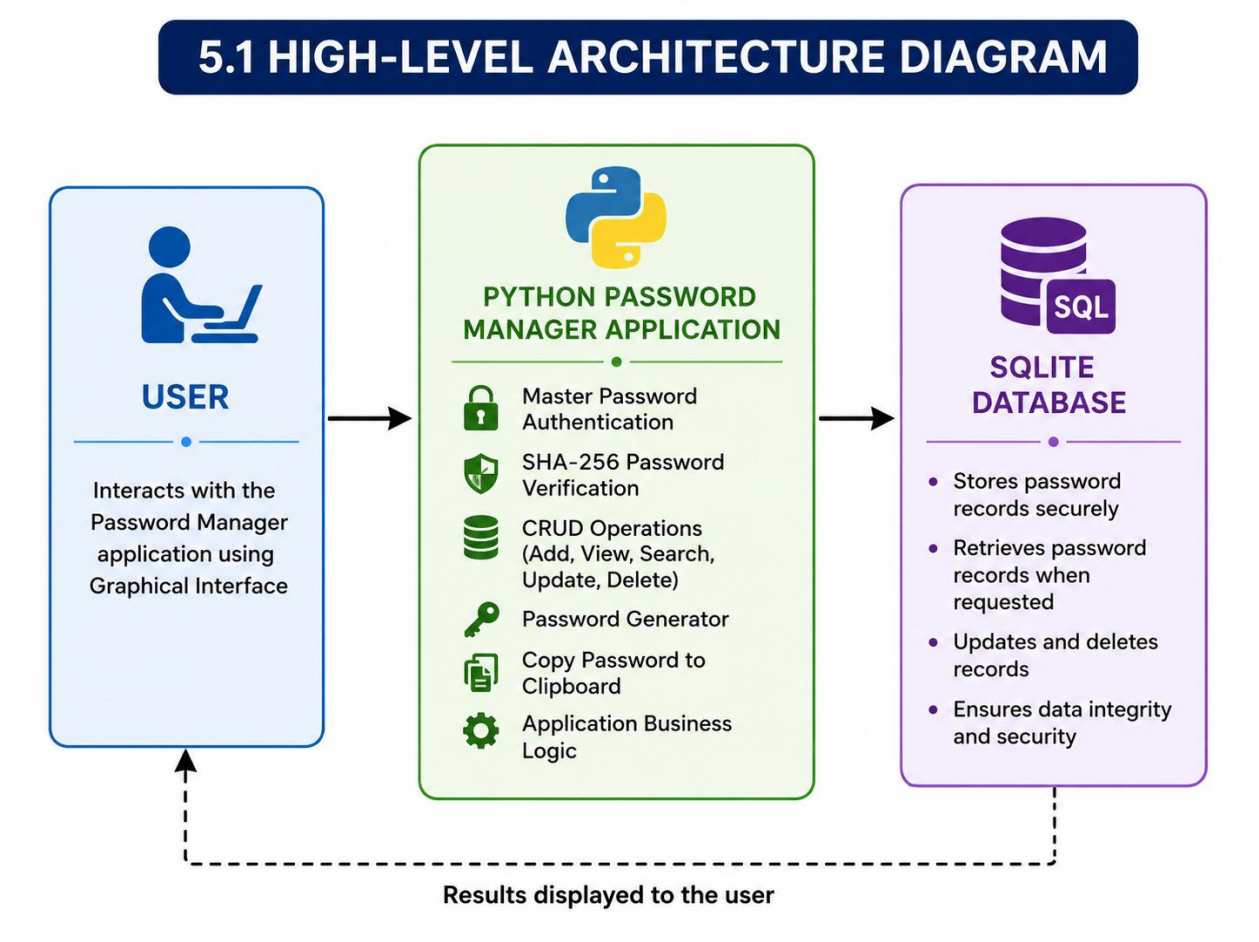
- User launches the application.
- Master password authentication is performed.
- User accesses the main dashboard after successful login.
- Password records can be added, viewed, searched, updated, or deleted.
- The password generator creates strong passwords when required.
- All data is securely stored and managed in the SQLite database.
- The application exits safely after completing user operations.

The modular design improves security, simplifies maintenance, and allows future enhancements without affecting the overall functionality of the application.

5.2 High Level Diagram

2 Description

The high-level architecture of the Password Manager consists of three layers:



1. **User**

- Interacts with the application using the keyboard.

2. **Python Application**

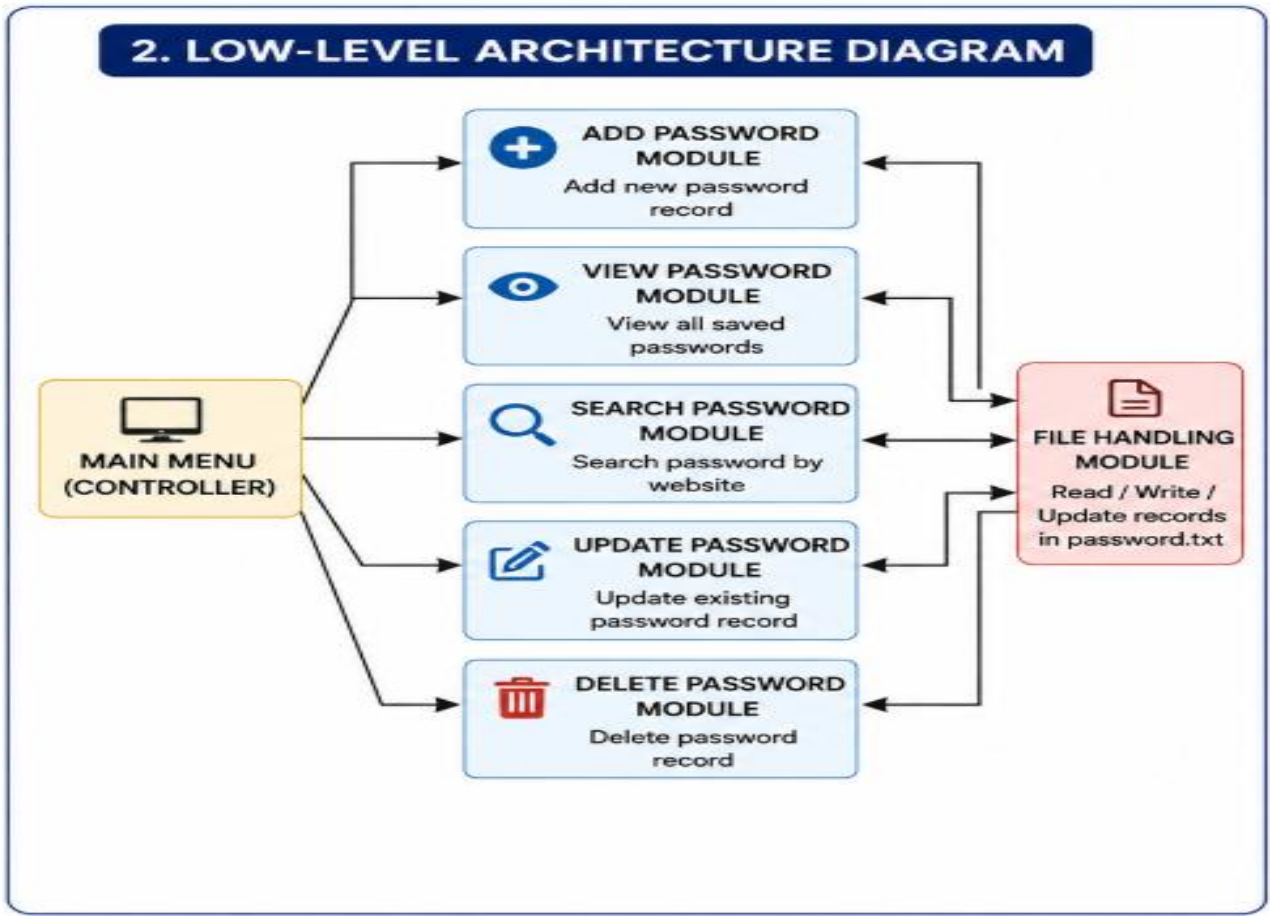
- Accepts user input.
- Performs validation.
- Executes CRUD operations.
- Password Generator
- Copy Password to clipboard
- Application business logic.

2. SQLite Database

- Stores password records securely.
- Retrieves password records on request.
- Updates and deletes existing records
- Maintain data integrity and persistence

This layered architecture keeps the application simple, modular, and easy to maintain.

5.3 Low Level Diagram



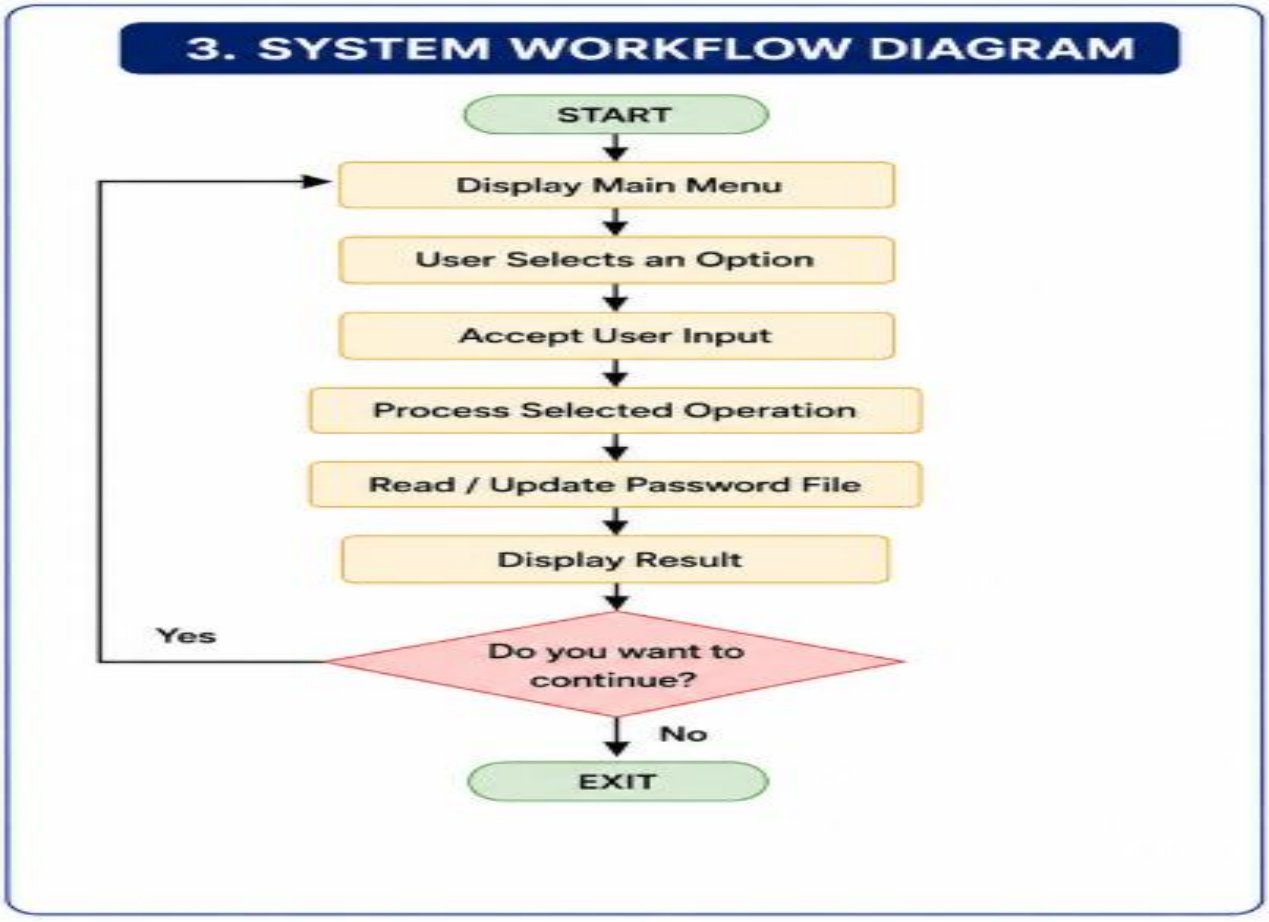
3 Description

The internal workflow of the application consists of the following modules:

- Main Menu
- Add Password Module
- View Password Module
- Search Password Module
- Update Password Module
- Delete Password Module
- File Handling Module

Each module performs one specific task and communicates with the password storage file.

5.4 System Workflow

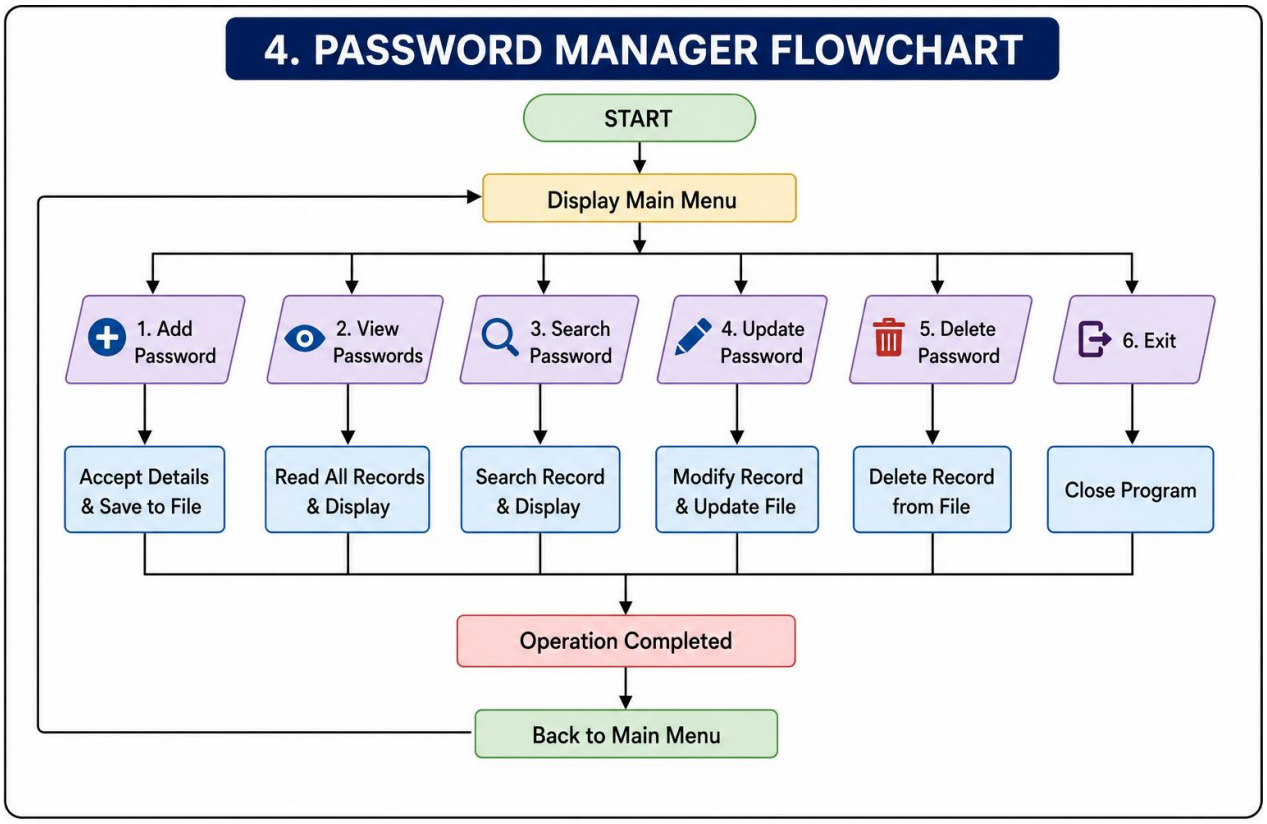


4 Workflow Steps

1. Start the application.
2. Display the main menu.
3. User selects an option.
4. Accept user input.
5. Process the selected operation.
6. Read or update the password file.
7. Display the output.
8. Return to the main menu.
9. Exit the application.

This workflow ensures smooth execution of every operation.

5.5 Flowchart



- **Flow Description**

The flowchart represents the logical execution of the program.

- Start
- Display Menu
- Read User Choice
- Perform Selected Operation
- Display Result
- Return to Menu
- Exit

5.6 Algorithm

- **Algorithm for Password Manager**

Step 1: Start the application.

Step 2: Initialize the SQLite database and create tables if they do not exist.

Step 3: Display the Master Password Login screen.

Step 4: Accept the master password from the user.

Step 5: Verify the entered password using the SHA-256 hashing algorithm.

Step 6: If the password is incorrect, display an error message and return to **Step 4**.

Step 7: If authentication is successful, open the Password Manager Dashboard.

Step 8: Display the available operations:

- Add Password
- View Passwords
- Search Password
- Update Password
- Delete Password

- Generate Password
- Copy Password
- Exit

Step 9: Accept the user's selected operation.

Step 10: Execute the selected operation and perform the corresponding SQLite database action.

Step 11: Display the operation result to the user.

Step 12: Ask the user whether they want to perform another operation.

Step 13: If **Yes**, return to **Step 8**; otherwise, proceed to **Step 14**.

Step 14: Close the application safely.

Step 15: End the program.

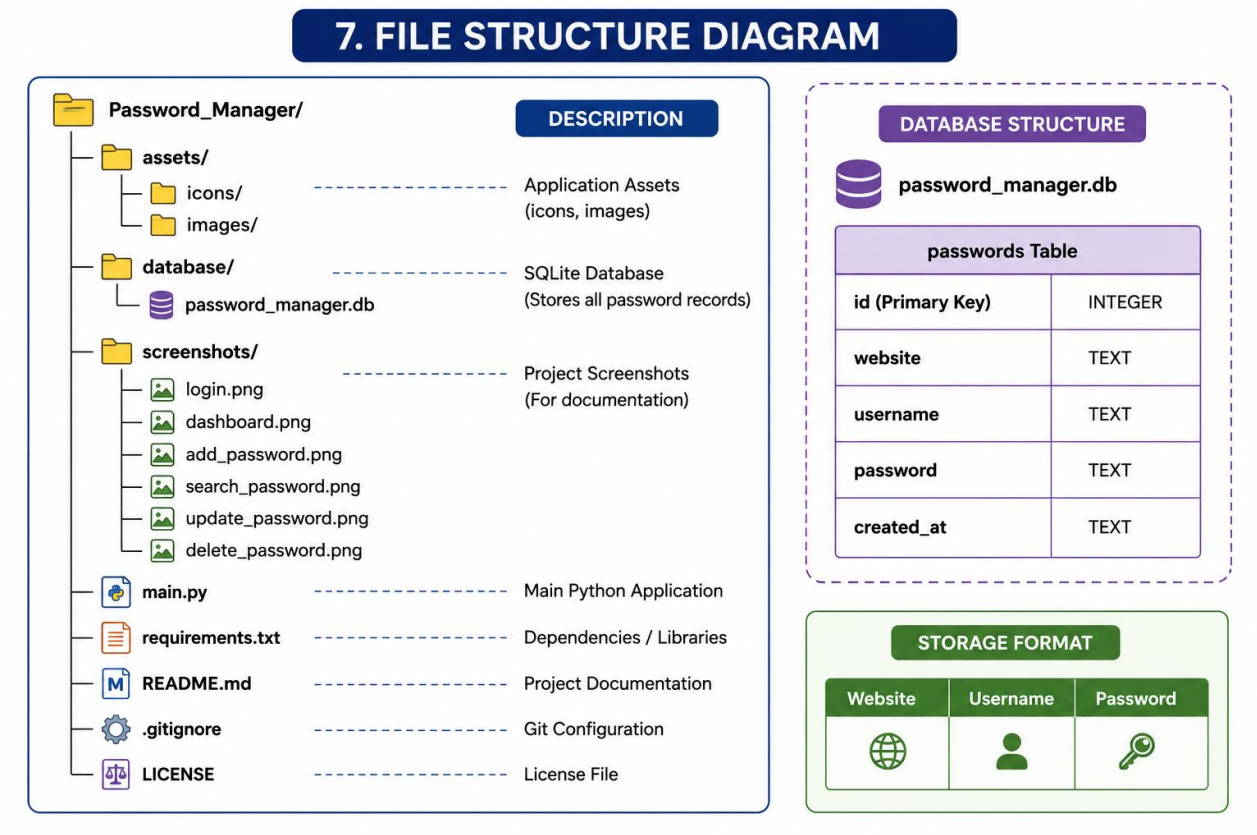
5.7 Advantages of the Proposed Design

The proposed design provides several benefits:

- Secure user authentication
- Protects data using SHA-256
- Reliable SQLite database storage
- Simple and intuitive interface
- Modular and scalable design
- Supports complete CRUD operations
- Fast password retrieval
- Strong password generation
- Offline functionality
- Easy maintenance and updates
- Lightweight and efficient
- Enhances data security
- Improves user productivity
- Easy future enhancements
- Cost-effective solution

5.8 Project Architecture

The Password Manager follows a modular architecture.



This architecture clearly separates user interaction, program logic, and data storage.

5.10 Summary

The **Password Manager** is a secure and user-friendly desktop application developed using **Python**, Tkinter, and SQLite to simplify password management. It enables users to securely store and manage their login credentials through features such as **master password authentication**, SHA-256 hashing, password generation, and complete **CRUD operations**. The application follows a modular architecture with an intuitive graphical interface, ensuring efficient performance, reliability, and ease of use. Its organized project structure, secure database management, and offline functionality make it a practical solution for protecting sensitive information while demonstrating the application of secure software development principles.

6. Performance Test

Testing is one of the most important phases of software development. It ensures that the application performs all expected operations correctly without errors. During the development of the Password Manager, different test cases were executed to verify the functionality of each module.

The project was tested in the Visual Studio Code terminal using different user inputs to ensure the correctness of CRUD (Create, Read, Update, Delete) operations.

6.1 Test Plan

The objective of testing was to verify that every feature of the Password Manager works correctly.

- **Test Environment**

Parameter	Details
Operating System	Windows 11
Programming Language	Python 3.x
IDE	Visual Studio Code
Database	SQLite
Version Control	Git & GitHub

6.2 Test Cases

Test Case ID	Module	Input	Expected Result	Status
TC-01	Master Login	Correct Master Password	User successfully logged in	Pass
TC-02	Master Login	Incorrect Master Password	Error message displayed	Pass

Test Case ID	Module	Input	Expected Result	Status
TC-03	Add Password	Website Name, Username &Password	Password stored d successfully in SQlite Databaseuccessfull	Pass
TC-04	Show/Hide Password	Toggle Show Password	Password visibility changed correctly	Pass
TC-05	Update Password	Modify existing password	Database updated successfully	Pass
TC-06	Delete Password	Delete selected record	Record removed successfully	Pass
TC-07	Exit	Exit option	Program closed successfully	Pass

6.3 Test Procedure

The following steps were followed while testing the application:

3.1.1 Test 1: Master Login

- Launch the Password Manager.
- Enter the master password.
- Click **Login**.
- Verify that the application opens successfully for a valid password.

Result: Pass

3.1.2 Test 2: Add Password

- Select **Add Password**.
- Enter the website name.
- Enter the username.
- Enter the password.
- Click **Save**.

- Verify that the password is stored successfully in the SQLite database.

Result: Pass

3.1.3 Test 3: Update Password

- Select an existing password record.
- Modify the required details.
- Click **Update**.
- Verify that the updated information is saved in the database.

Result: Pass

3.1.4 Test 4: Delete Password

- Select a password record.
- Click **Delete**.
- Confirm the deletion.
- Verify that the selected record is removed from the database.

Result: Pass

3.1.5 Test 5: Password Generator

- Click **Generate Password**.
- Verify that a strong random password is generated.

Result: Pass

3.1.6 Test 6: Show/Hide Password

- Click the **Show Password** option.
- Verify that the password visibility changes correctly.
- Click again to hide the password.

Result: Pass

3.1.7 Test 7: Copy Password

- Select a saved password.
- Click **Copy Password**.
- Verify that the password is copied to the clipboard.

Result: Pass

3.1.8 Test 8: Exit Application

- Click the **Exit** button.
- Verify that the application closes without any errors.

Result: Pass

6.4 Performance Outcome

The Password Manager performed successfully during testing. All CRUD operations executed correctly without runtime errors.

The application successfully:

- All application modules executed successfully without errors.
- CRUD operations were performed accurately using the SQLite database.
- Master password authentication worked correctly.
- Password generation and clipboard functions responded instantly.
- Search, update, and delete operations returned accurate results.
- Input validation prevented invalid or empty data entry.
- The Tkinter GUI remained responsive throughout testing.
- Database operations were completed with minimal response time.

- No data loss or application crashes were observed during testing.
- Overall application performance met the expected functional and security requirements.

6.5 Result Analysis

The testing results indicate that the Password Manager meets all the functional requirements specified during project planning.

- All functional requirements were successfully implemented.
- Master password authentication worked correctly.
- Password records were stored and retrieved accurately from the SQLite database.
- CRUD (Create, Read, Update, Delete) operations executed successfully.
- Password generation produced strong and random passwords.
- Input validation handled invalid and empty entries effectively.
- The graphical user interface was responsive and user-friendly.
- No data inconsistency or application crashes were observed during testing.
- The application met the expected security and performance objectives.
- Overall testing confirmed that the Password Manager is reliable, efficient, and ready for use.

No critical issues were observed during testing.

6.5 Advantages of Testing

Testing provided the following benefits:

- Ensures that all application features work correctly.
- Identifies and fixes errors before deployment.
- Verifies the accuracy of CRUD operations.
- Improves the reliability and stability of the application.
- Ensures proper validation of user inputs.
- Confirms secure storage and retrieval of passwords.
- Enhances the overall user experience.
- Reduces the chances of application failure.
- Increases user confidence in the software.
- Helps maintain the quality and performance of the application.

6.6 Limitations

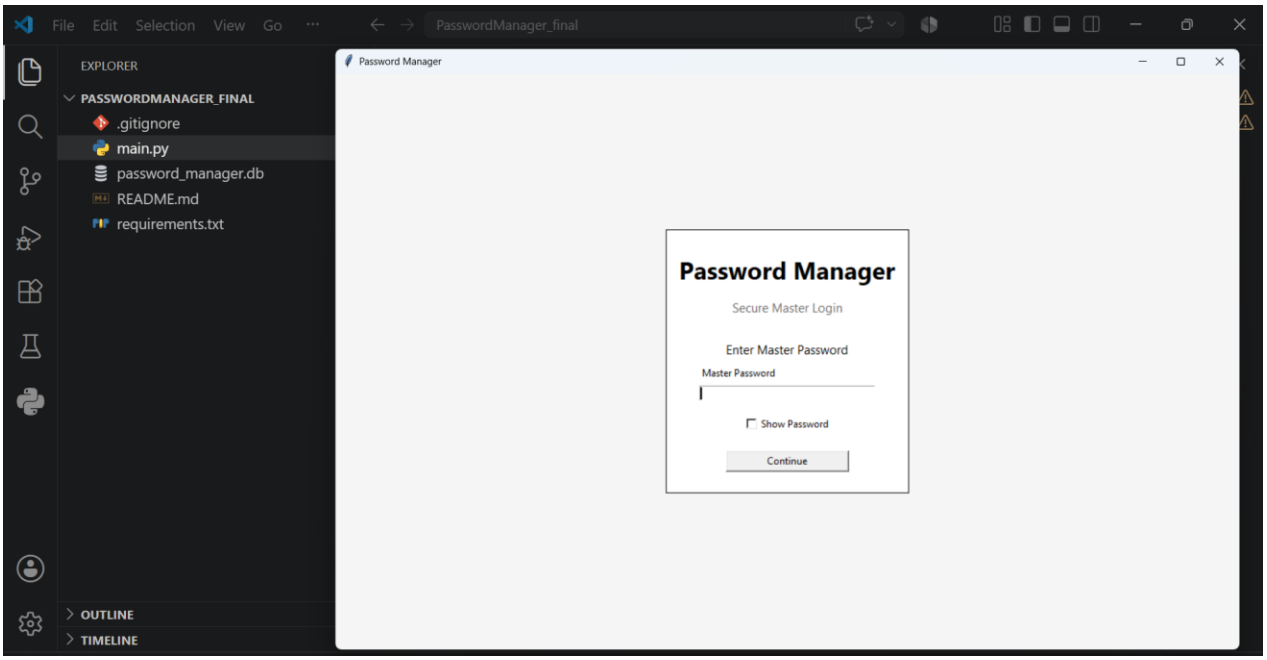
Although the Password Manager works successfully, it has some limitations:

- Testing was conducted only in the Windows environment.
- Limited test data was used during testing.
- Multi-user access was not tested.
- Cross-platform compatibility was not verified.
- Advanced security features were not included in testing.
- Long-term performance and stress testing were not performed.
- Testing was limited to the implemented project modules only

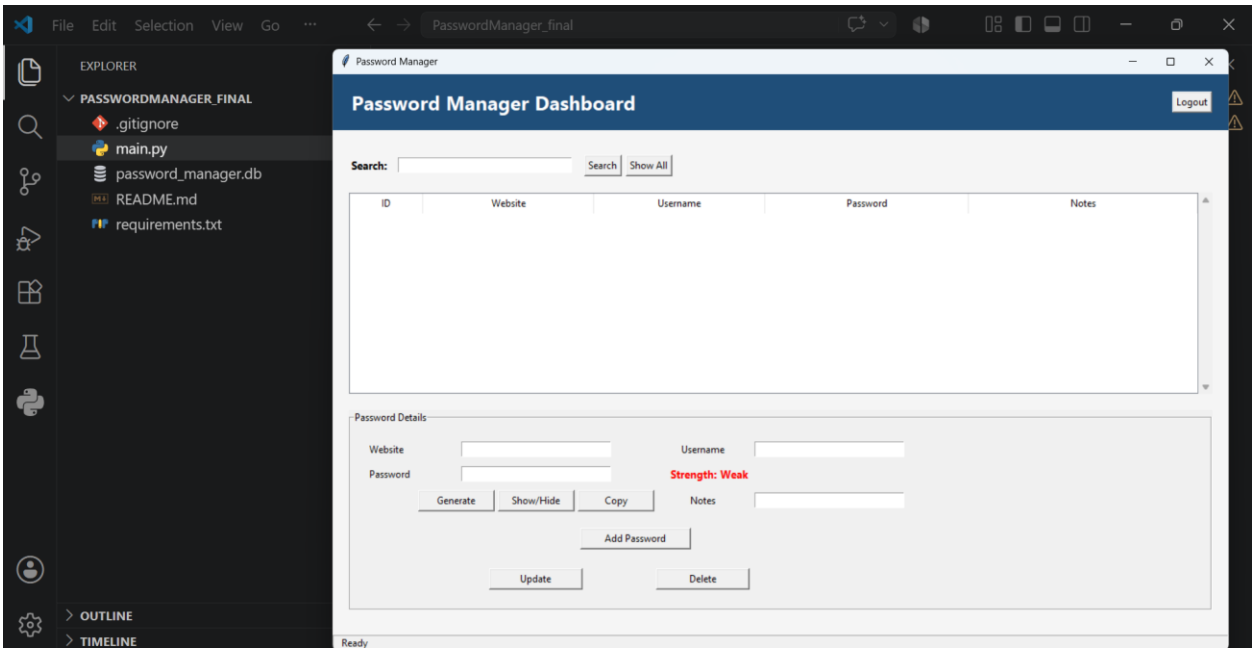
6.7 Screenshots

Insert the following screenshots in this section:

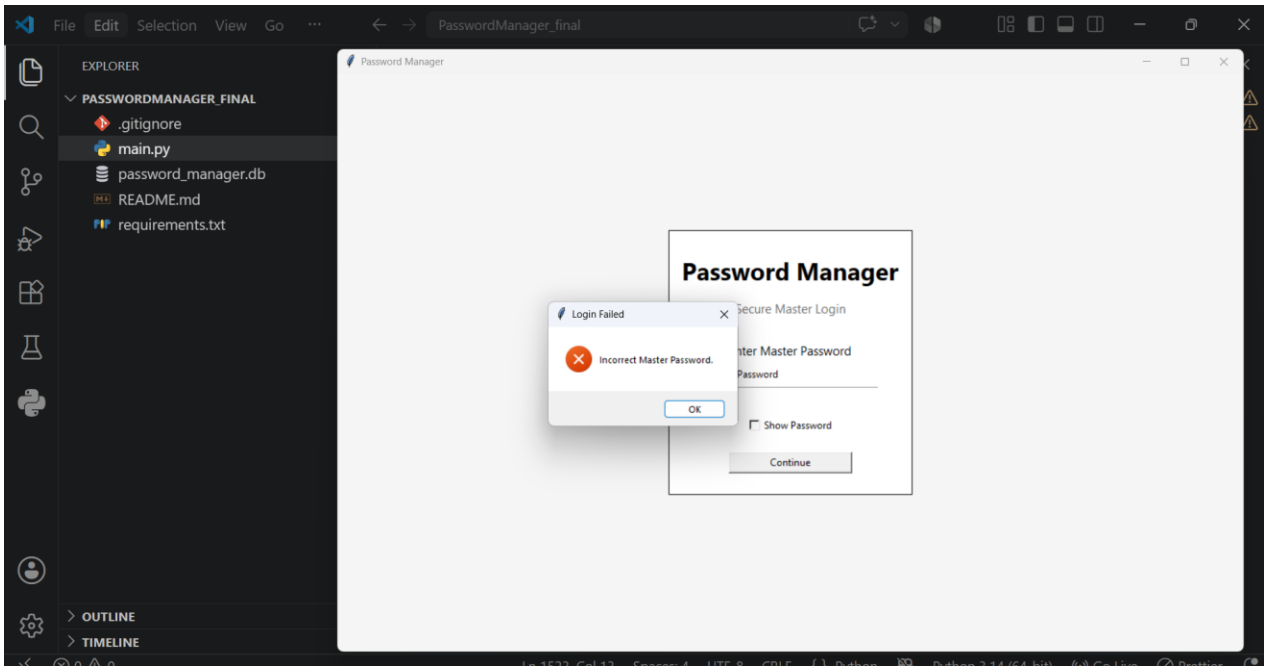
1. Password Manager login :



2. Login successfully

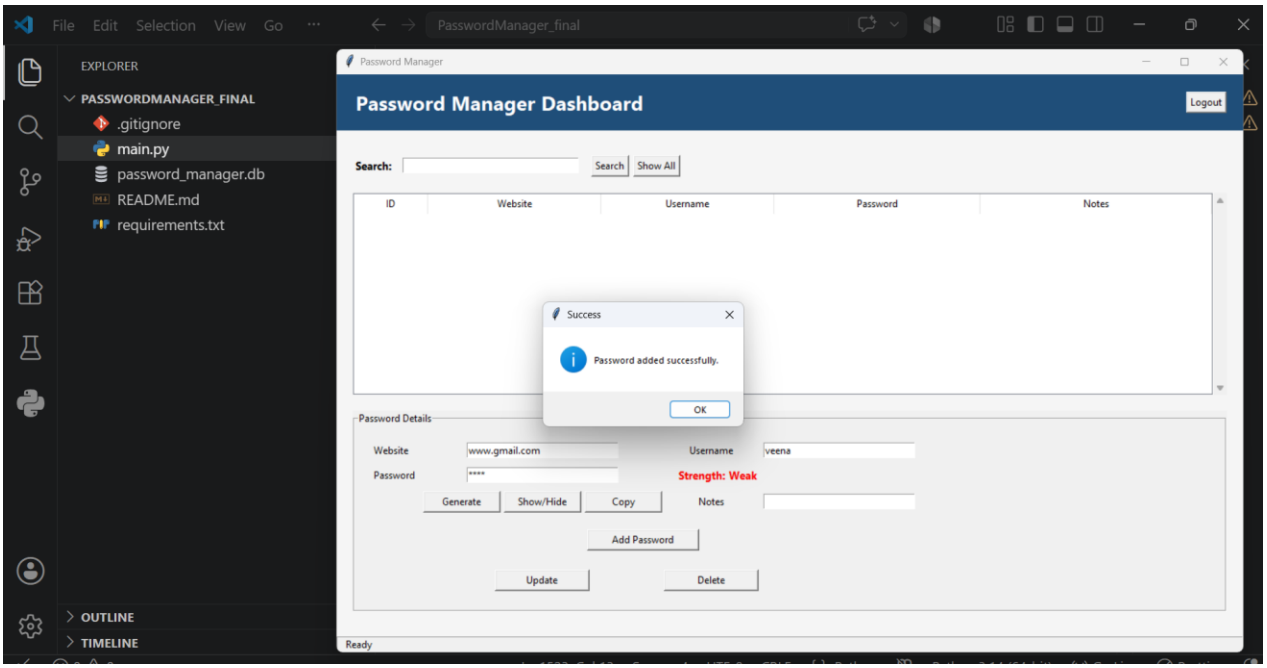


3. Login with wrong Password :

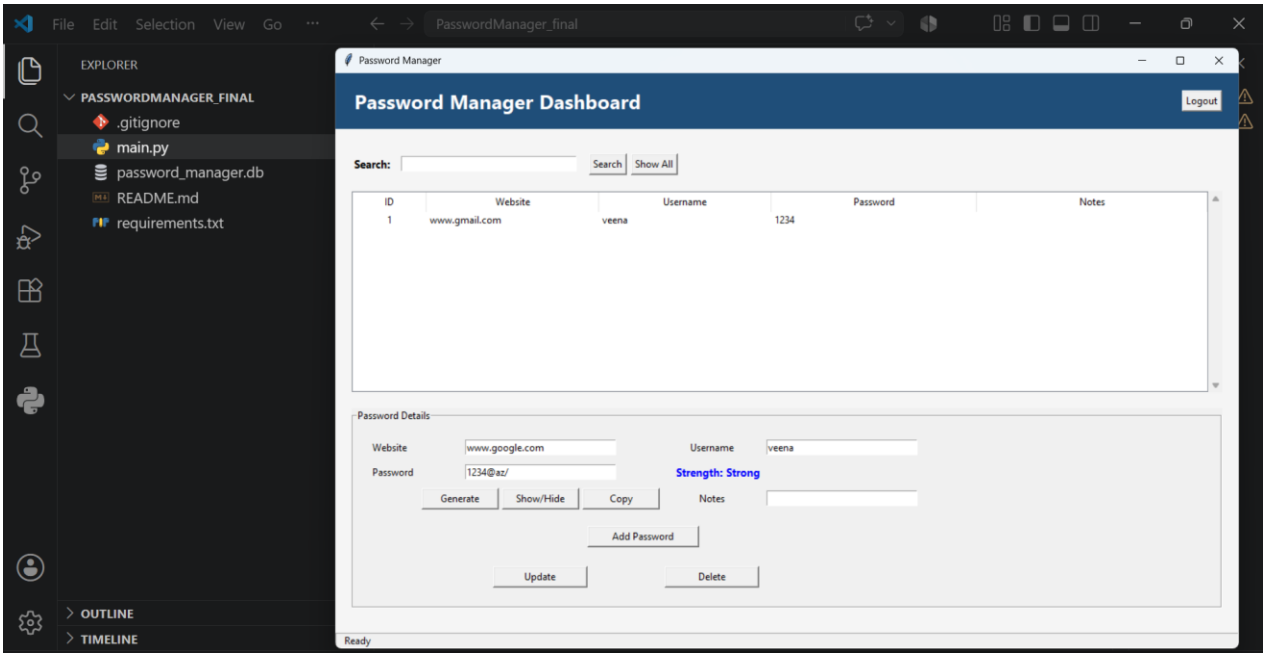


4.

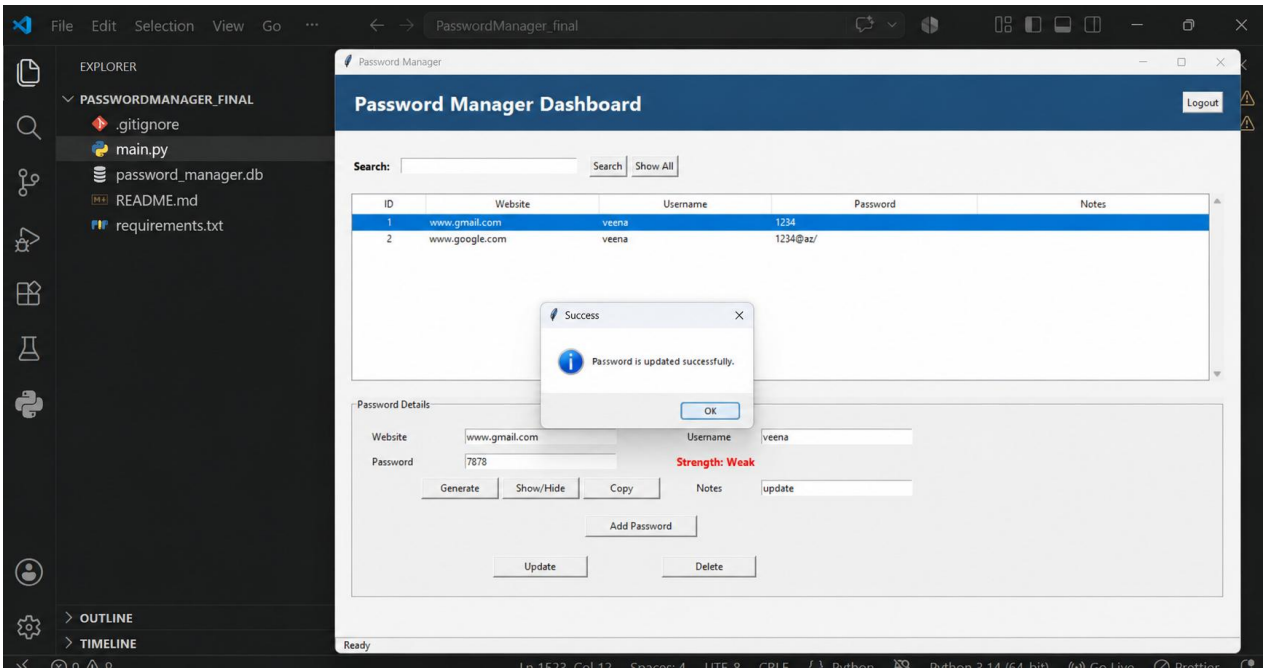
5. Add Password



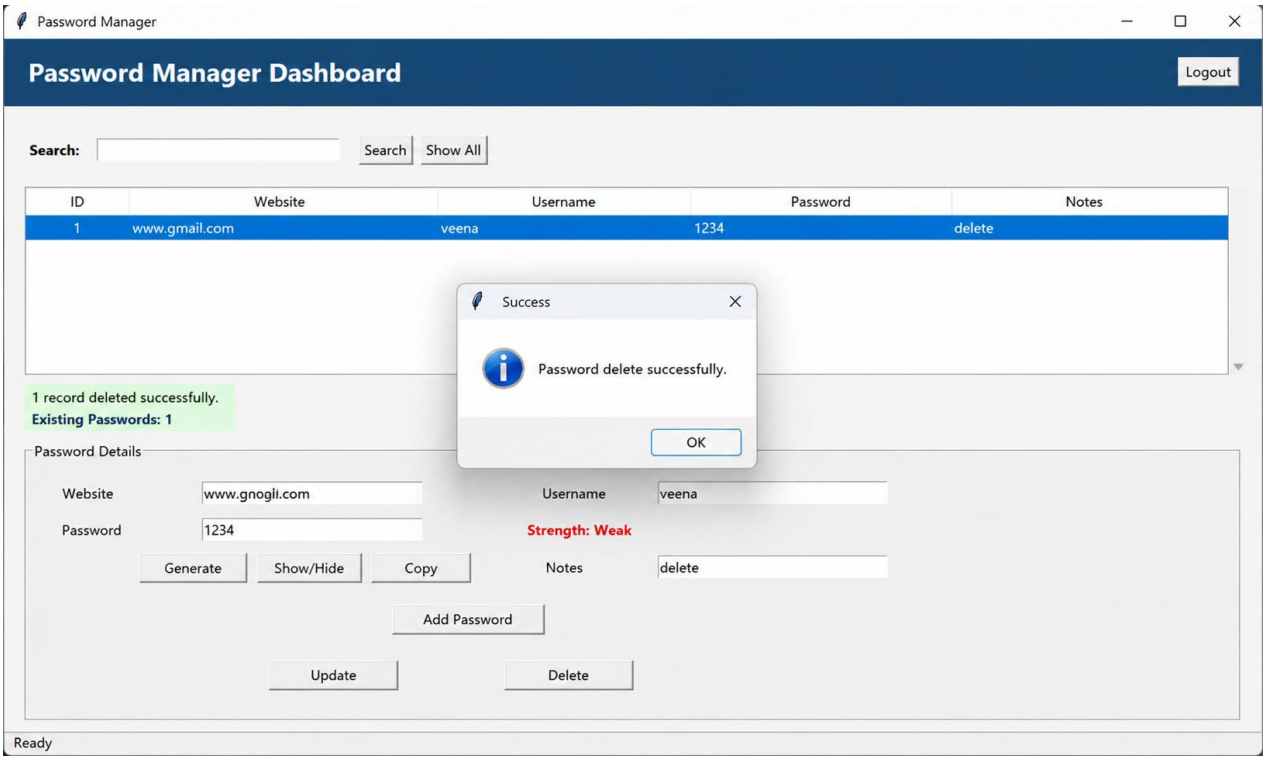
6. Show/Hide Passwords



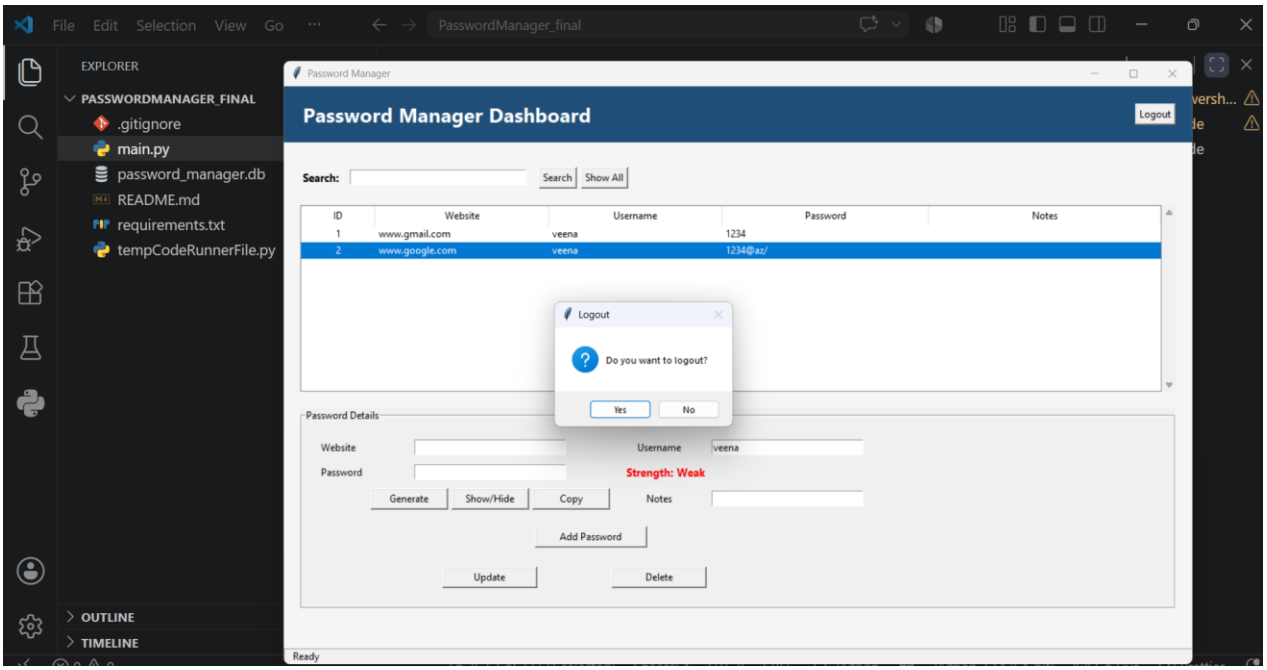
7. update Password



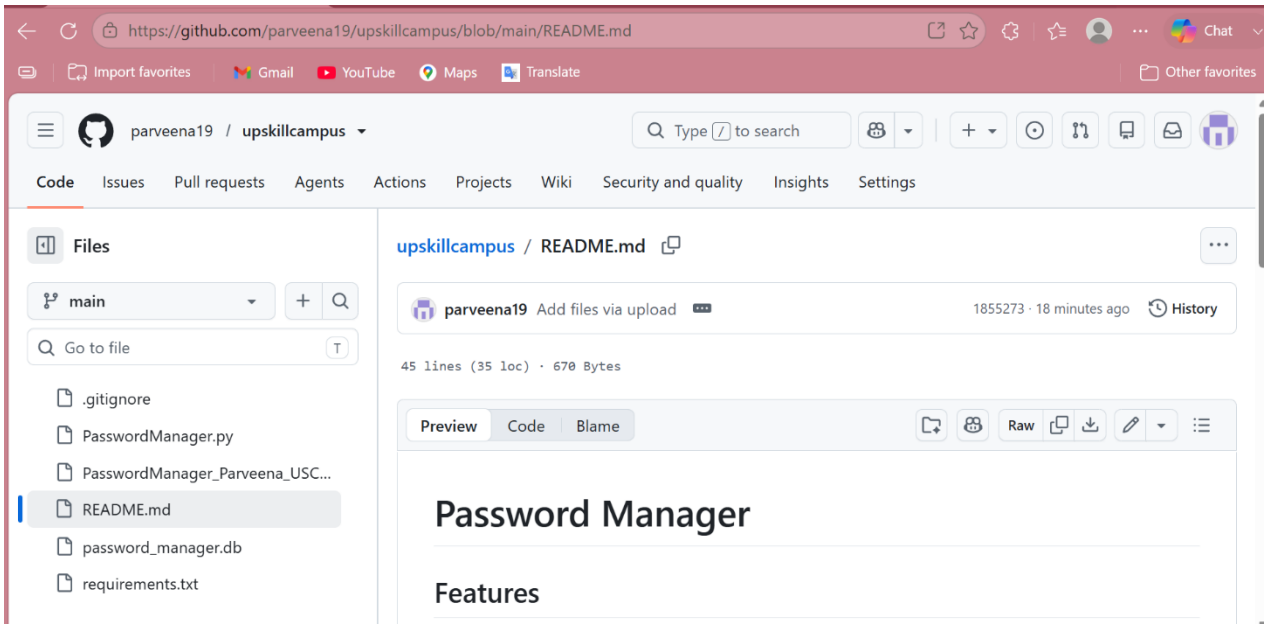
8. Delete Password



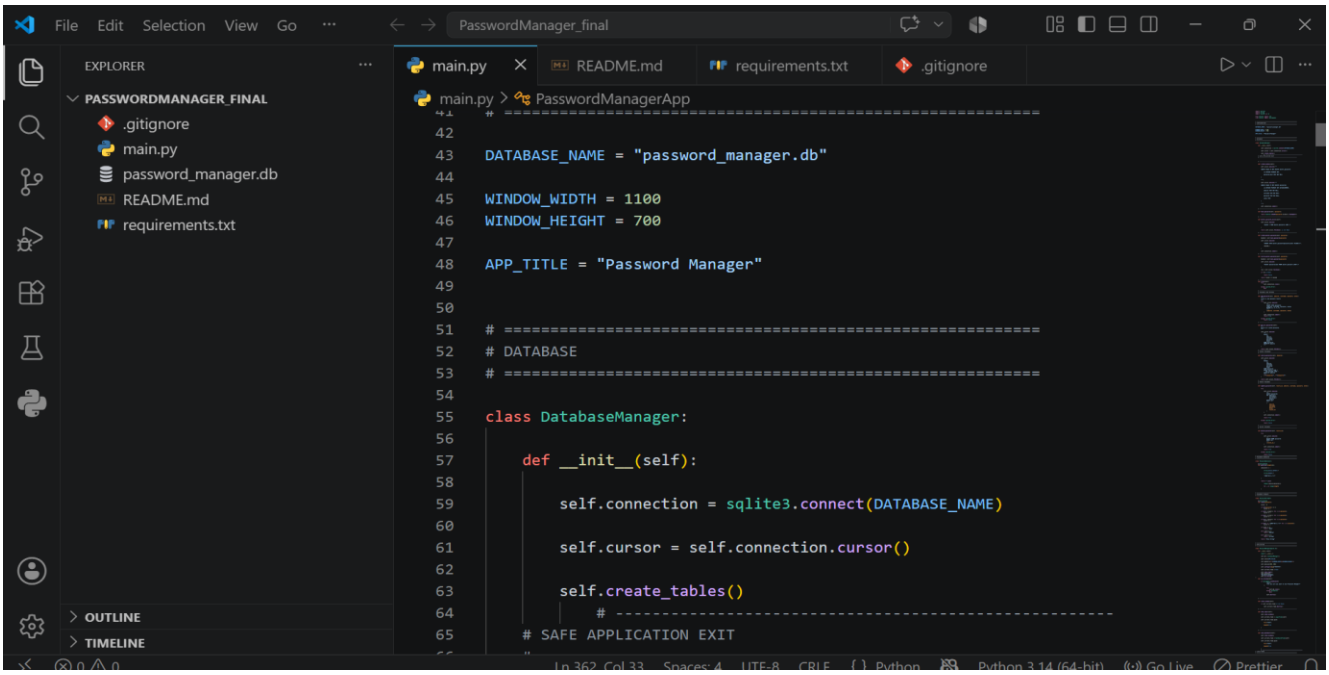
9. Logout :



10. GitHub Repository :



11. VS Code Project Structure



6.9 Testing Conclusion

The testing process confirmed that the Password Manager application meets its functional and performance requirements. All major features, including user authentication, password management, database operations, and password generation, worked correctly. The application produced accurate results, handled invalid inputs effectively, and maintained stable performance throughout testing. Overall, the software is reliable, secure, and ready for deployment.

7. My Learnings

The Python Development Internship provided valuable practical experience in software development. During the internship, I improved my programming knowledge by implementing a real-world project using Python.

The major learnings include:

- Improved Python programming skills.
- Learned GUI development using Tkinter.
- Gained experience with SQLite database management.
- Understood CRUD operations implementation.
- Learned password hashing using SHA-256.
- Improved debugging and testing skills.
- Enhanced knowledge of secure password management.
- Strengthened problem-solving and logical thinking.
- Improved code organization using OOP concepts.
- Gained practical experience in the Software Development Life Cycle (SDLC)

The internship helped me gain confidence in developing software applications independently.

8. Skills Gained

During the internship, I developed the following technical and professional skills:

- **Technical Skills**
- Python Programming
- Object-Oriented Programming (OOP)
- GUI development using Tkinter
- SQLite database management
- CRUD Operations
- Git
- GitHub
- Visual Studio Code
- **Soft Skills**
- Problem Solving
- Time Management
- Documentation
- Self Learning
- Analytical Thinking
- Communication
- Project Planning

9. Future Work Scope

The Password Manager can be enhanced further by implementing additional features such as:

- Multi-user Support
- Add cloud synchronization for multi-device access.
- Implement two-factor authentication (2FA).
- Integrate biometric login (fingerprint/Face ID).
- Use AES encryption for enhanced password security.
- Add password breach detection and security alerts.
- Enable auto-fill for websites and applications.
- Support secure password backup and recovery.
- Provide password import and export functionality.
- Develop mobile and web application versions.
- Add password health analysis for weak and reused passwords

These improvements can make the application more secure, user-friendly, and suitable for real-world usage.

Conclusion

The Python Development Internship provided valuable practical exposure to software development using Python. During the internship, I successfully designed and developed a **Password Manager** application using **Python**. The application enables users to securely store and manage passwords through an intuitive graphical user interface while supporting CRUD operations and secure master password authentication.

The project enhanced my understanding of Python programming, GUI development, database management, software testing, debugging, version control using Git and GitHub, and documentation. It also strengthened my problem-solving abilities and provided hands-on experience in developing a secure and user-friendly desktop application.

Overall, the internship was a rewarding learning experience that enhanced both my technical knowledge and professional skills. It increased my confidence in developing real-world software applications and prepared me for future opportunities in software development.

References

The following resources were used during the development of this project:

1. Python Official Documentation
   https://docs.python.org/3/
2. Git Documentation
   https://git-scm.com/docs
3. GitHub Documentation
   https://docs.github.com/
4. Visual Studio Code Documentation
   https://code.visualstudio.com/docs
5. upskill Campus Learning Resources
6. UniConverge Technologies Pvt. Ltd. Training Material
7. Python File Handling Documentation